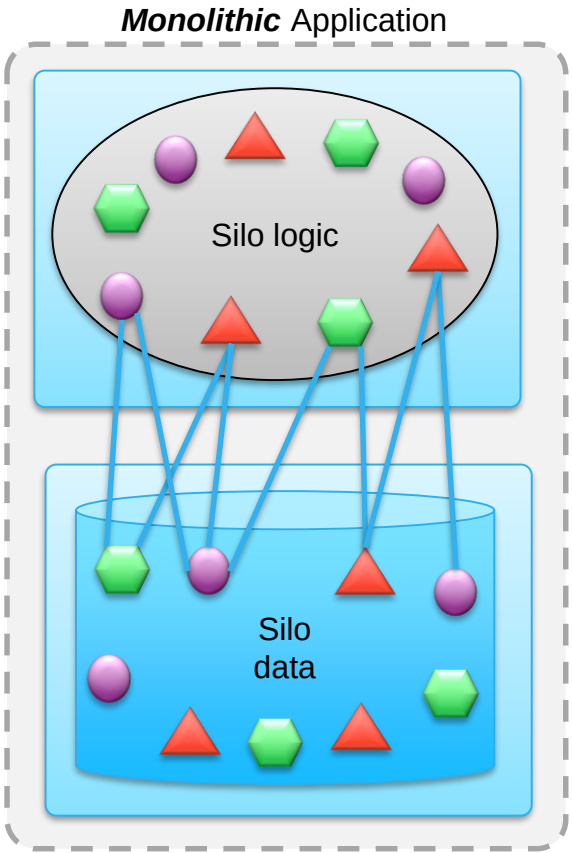
Microservices: How they relate to ESB, APIs and messaging

Kim Clark
Integration Architect
Offering Management for Hybrid Integration



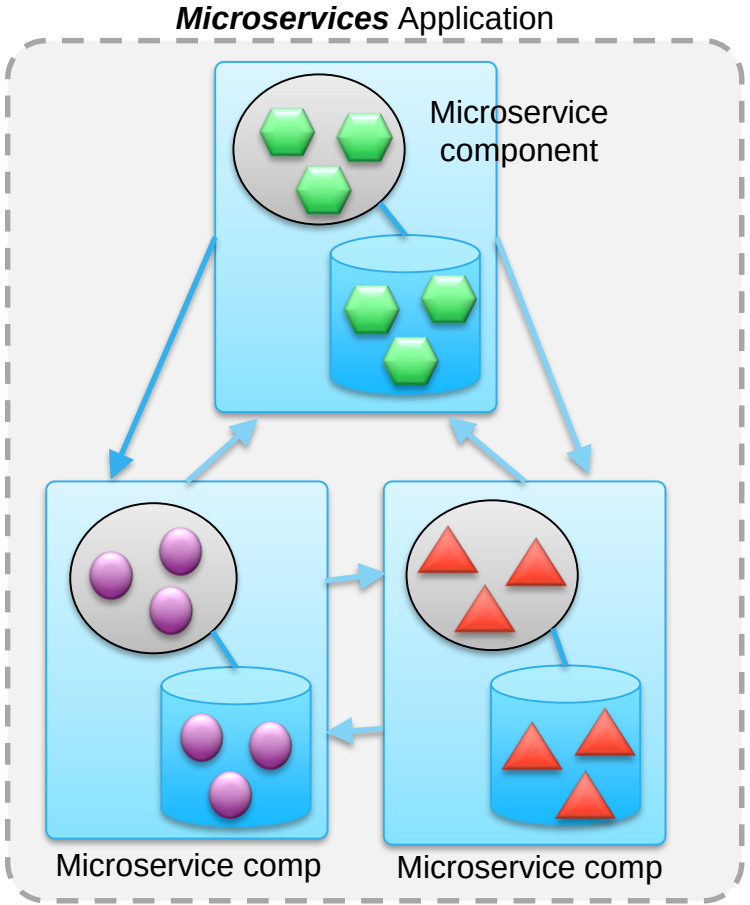
What *is* microservices architecture?



A set of principles that guide the breaking down of large components into smaller, more self contained ones enabling greater agility, more elastic scalability and and more differential resilience.

Fundamental principles include maximizing isolation and decoupling between components (including their data), making components as stateless as possible and avoiding affinities.

Supporting requirements include fully automated DevOps pipelines and good automated test coverage.



Why Microservices?

Agility

- Faster iteration cycles
- Bounded context

Scalability

- Elastic scalability
- Workload orchestration

Resilience

- Reduced dependencies
- Fail fast

What are the challenges?

Maturity
- Are you ready for a radical change in methods, skillsets, infrastructure, operations.
- Are you sufficiently automated (infrastructure, test, dev pipeline, deployment etc.)

Maintenance
- Will you be able to sustain the skillsets needed to maintain the microservices architecture in the future?

Latency & Serialization
- A request/response chained down a set of microservices must incur extra latency from network hops and serialization
- Serialization has advanced massively in recent years, but inevitably has some contribution to CPU usage

Data sharing
- Not all data can be split into neat independent functions. Some things are shared, and this needs careful design

Real-time dependencies and their combined availability
- Microservices calling other microservices synchronously need careful consideration
- Tends to creep, as one service builds on top of another
- Need to move to more complex message based techniques and/or introduce availability patterns such as circuit breaker

Manageability
- How do you manage and monitor a vast network of microservices
- How do you diagnose problems across a heavily distributed landscape

How does persistence work?
- Pessimistic versus Optimistic
- How to handle shared objects
- Relational / NoSQL
- ACID / BASE / CQRS / Event Sourcing?

Are you really doing microservices or just aligning with some of the microservice principles?

Martin Fowler/James Lewis

1. Componentization
2. Organized around Business Capabilities
3. Products not Projects
4. Smart endpoints and dumb pipes
5. Decentralized Governance
6. Decentralized Data Management
7. Infrastructure Automation
8. Design for failure
9. Evolutionary Design

http://martinfowler.com/articles/microservices.html

12 factor apps

I. Codebase
II. Dependencies
III. Config
IV. Backing Services
V. Build, release, run
VI. Processes
VII. Port binding
VIII. Concurrency
IX. Disposability
X. Dev/prod parity
XI. Logs
XII. Admin processes

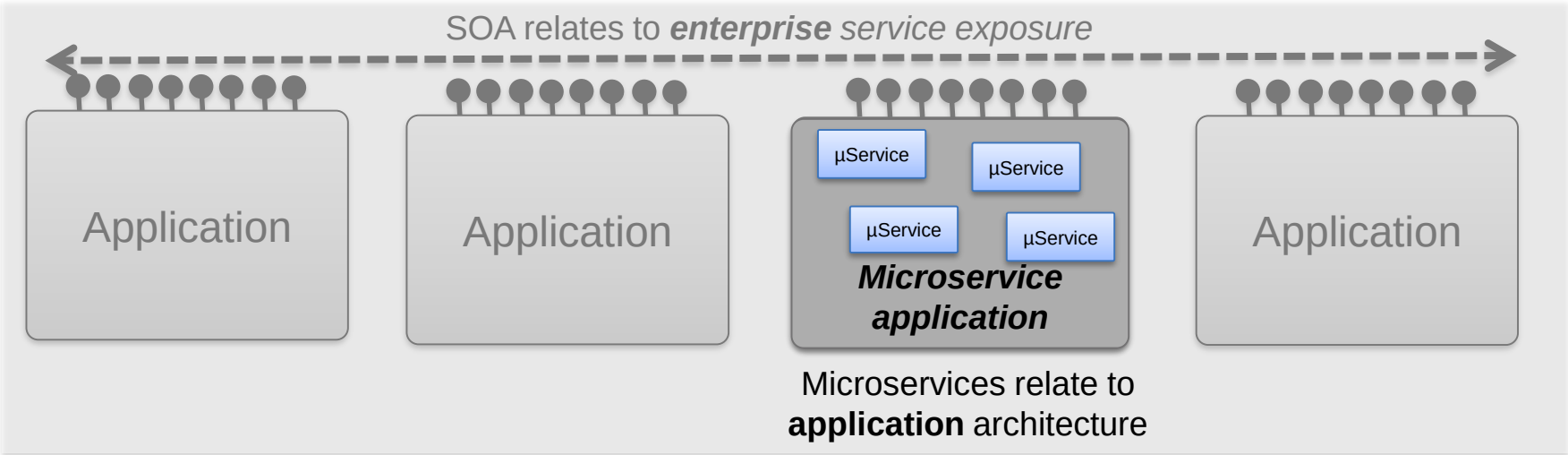
http://12factor.net



Containers, orchestration frameworks, event sourced applications, eventual consistency, CQRS, circuit breaker, bulkhead, service discovery, sidecars, routing fabrics, continuous delivery, agile programming, test driven development, contract driven development, domain driven design, centralised logging, polygot runtimes…...

Consider the adoption paths of SOA, XP, agile, devops etc. These often came with an “all or nothing” message, but can you take on the whole package?

Service oriented architecture (SOA) and **microservices architecture** relate to different scopes



This simple distinction can be contentious depending on your definition of SOA

Microservices vs SOA - short blog post and 10 min video
http://ibm.biz/MicroservicesVsSoaBlog

Original PoV paper on microservices and in integration
http://ibm.biz/MicroservicesVsSoa (~ 15 pages)
webinar based on above paper
http://ibm.biz/MicroservicesVsSoaFullWebinar (55 mins)

Some **microservices principles** are *really* different to SOA

Reuse is not the goal

Re-use of common components is discouraged due to the dependencies it creates. Re-use by copy is preferred.

Synchronous is bad

Making synchronous calls such as API or web services creates real-time dependencies. Messaging is used wherever possible between microservices.

Client side load balancing

Components are assumed to be volatile, so it is often the client's responsibility to find and even load balance across instances.

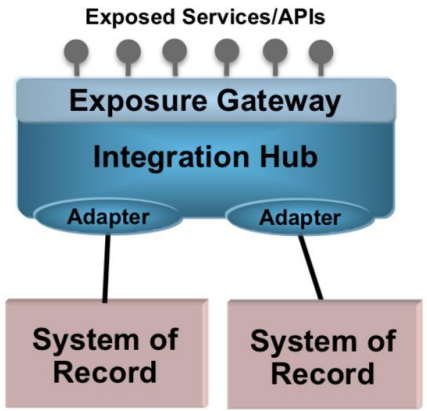
Data duplication is embraced

Techniques such as event sourcing result in multiple independent "views" of the data, ensuring the microservices are truly decoupled.

What was SOA really about?
Integration or **Components**?

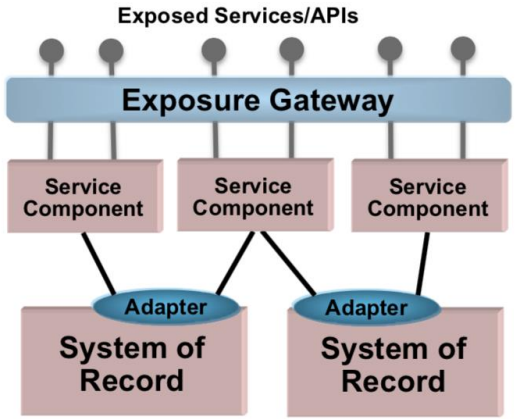
"SOA is about how to achieve **integration** often to aging complex back end systems in order to expose services"

In this case SOA is primarily a connectivity problem with little relationship to microservices architecture and certainly at a different scope.

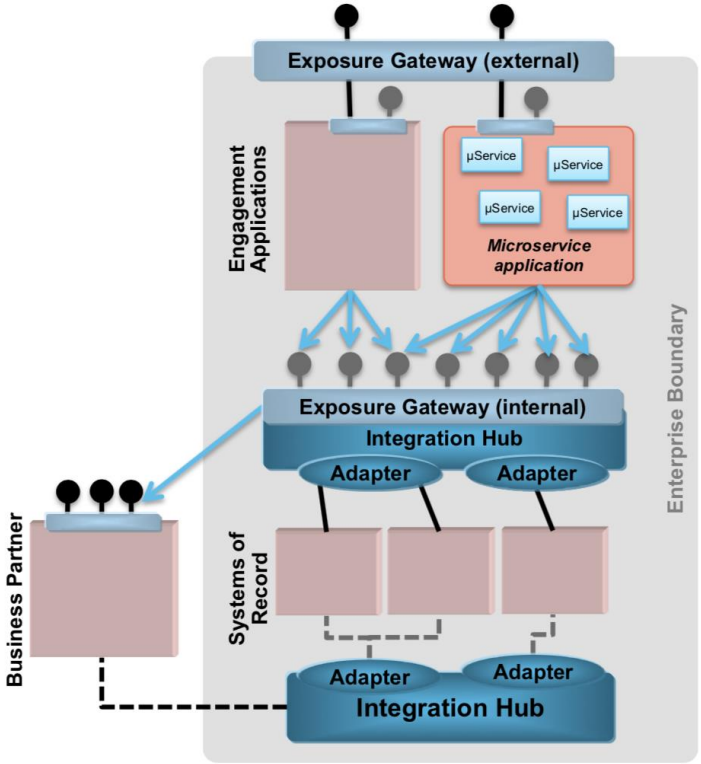


"SOA is about re-factoring your IT landscape into **components** that better align with the business needs and expose the services that it needs"

Here the connectivity problem is pushed down to the applications and the focus of SOA is on realignment of to the business needs. The service components look more like applications, and we might consider microservices as "more granular SOA", or even "SOA done right".



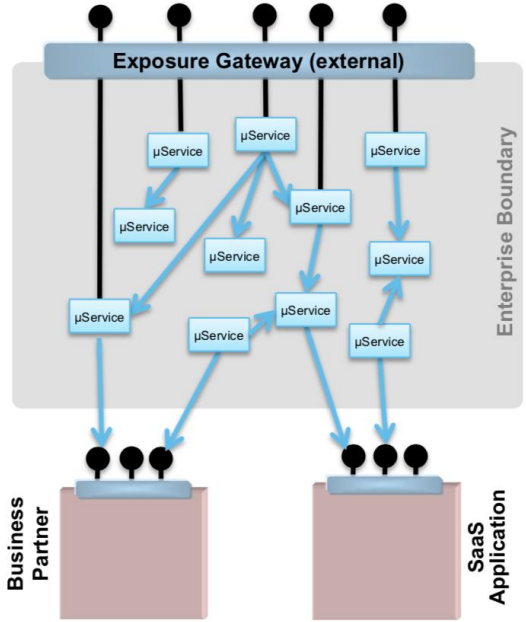
Why such split opinions on microservices vs SOA?



Mature large enterprise

Microservices are just one style of application

Exposing services is an *integration and data* challenge



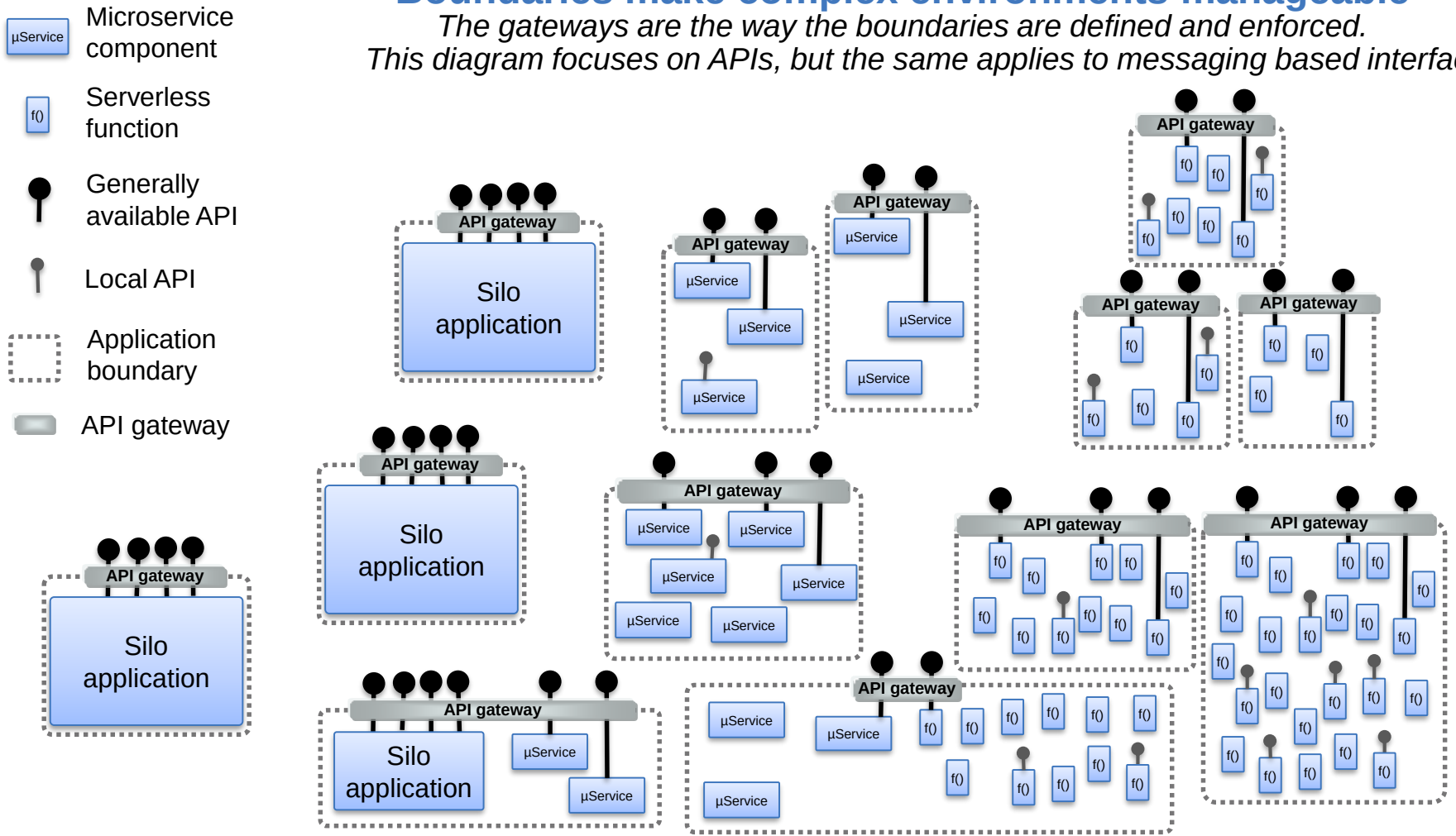
Green field online start-up

Much of landscape could be microservice based

The landscape *is* as (micro)service oriented architecture

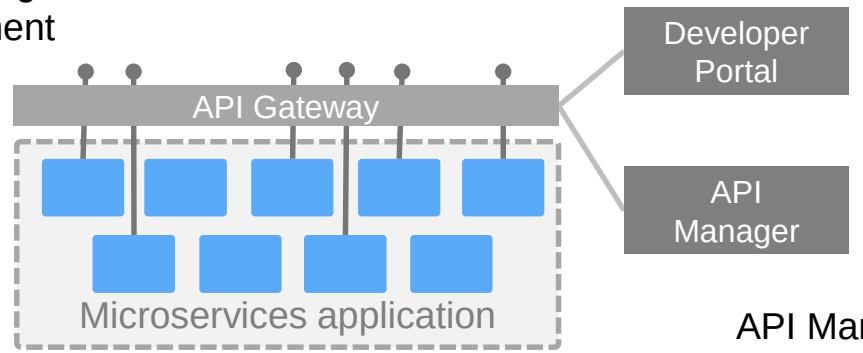
Boundaries make complex environments manageable

The gateways are the way the boundaries are defined and enforced.
This diagram focuses on APIs, but the same applies to messaging based interfaces.



Importance of API management for microservices

API Gateway:
- Decoupling/routing
- Traffic management
- Security
- Translation



Developer portal:
- API discovery
- Self subscription/administration
- Account usage analytics

API Manager:
- Plan/product design
- Access management
- Policy administration
- API plan usage analytics

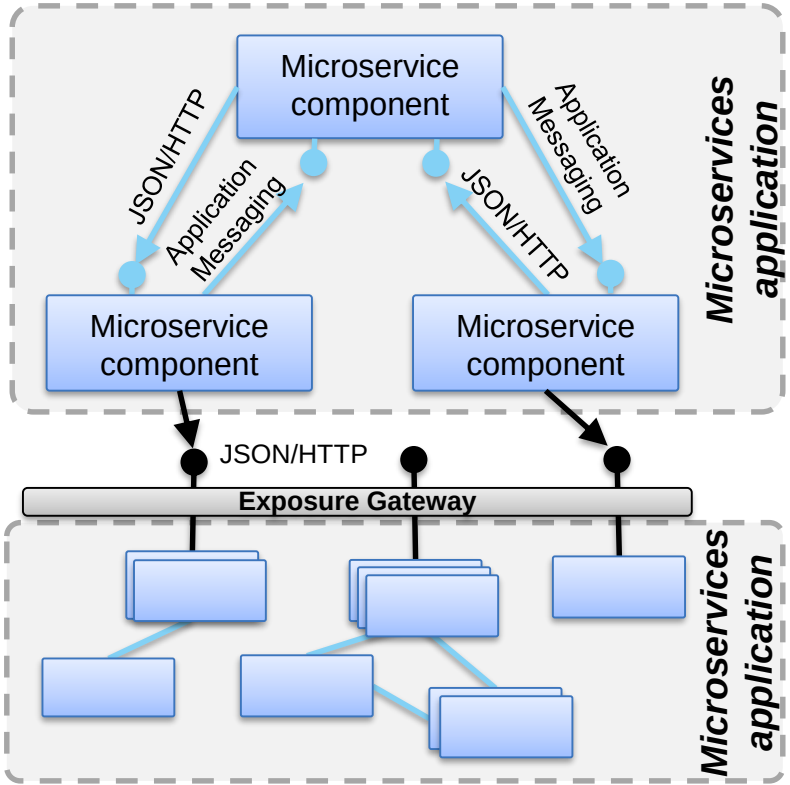
Individual microservice components should not be burdened with the complexities of API exposure beyond the microservices application boundary. Exposure should be delegated to a separate capability providing as a minimum, a gateway, a developer portal, and API management.

Inter-microservice vs. inter-application communication

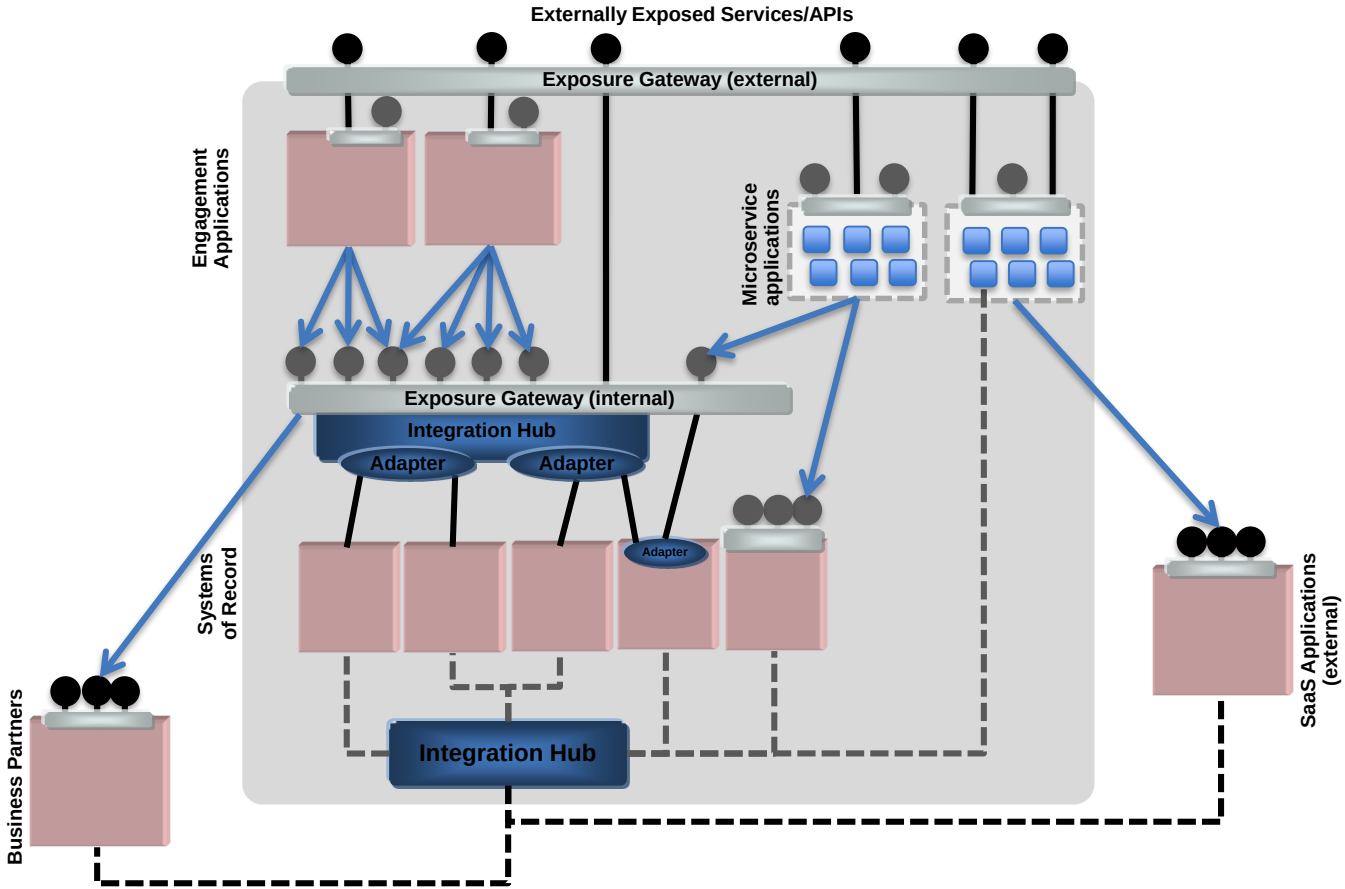
Inter-**microservice** communication
- Lightweight protocols: HTTP, application messaging
- Runtime component registry
- Client-side load balancing and circuit breaker patterns

Inter-**application** communication
- Enterprise protocols: Managed API gateways, enterprise messaging
- Design time developer portals
- Gateway load balancing and throttling

JSON/HTTP RESTful communication styles may be present in both types of communication, but their implementation may be radically different.



Integration in the real world today



Caution with comparisons (or “how to start an argument”!)

Comparisons that make some level of sense

Microservice architecture vs. Service oriented architecture

Trying to achieve many of the same things (enable the creation of re-usable business functions as isolated components), but perhaps a different scale. SOA evolved to become focused on the enterprise scale (though some might say that wasn’t the original intent). Microservices, is primarily at the application scale.

Microservice component vs. Service component

Both describe the actual implementation of a business function. They differ primarily in granularity, but also arguably in maintainability, agility etc.

Confusing comparisons

~~Microservices vs. SOA~~

Microservices are components. SOA is an architecture. What you’re probably trying to compare is Microservices Architecture with SOA (see above).

~~Microservices vs. APIs~~

APIs are a **interface**. A mechanism for *exposing* business function. Microservices are the **component** architecture used to *implement* those business functions. It makes no sense to directly compare them.

~~Microservice vs. Service~~

“Service” means different things to different people. Needs to be qualified. Service Implementation? Service exposure? Service definition? The same could be said for microservice. Need to be specific.

Looking for more information?

Moving to agile integration architecture
http://ibm.biz/AgileIntegArchPaper

Further links on agile integration architecture:
http://ibm.biz/AgileIntegArchLinks

Source paper on **microservices in relation to SOA**
http://ibm.biz/MicroservicesVsSoa
Webinar based on above paper
http://ibm.biz/MicroservicesVsSoaFullWebinar

Look out for related posts and videos on:

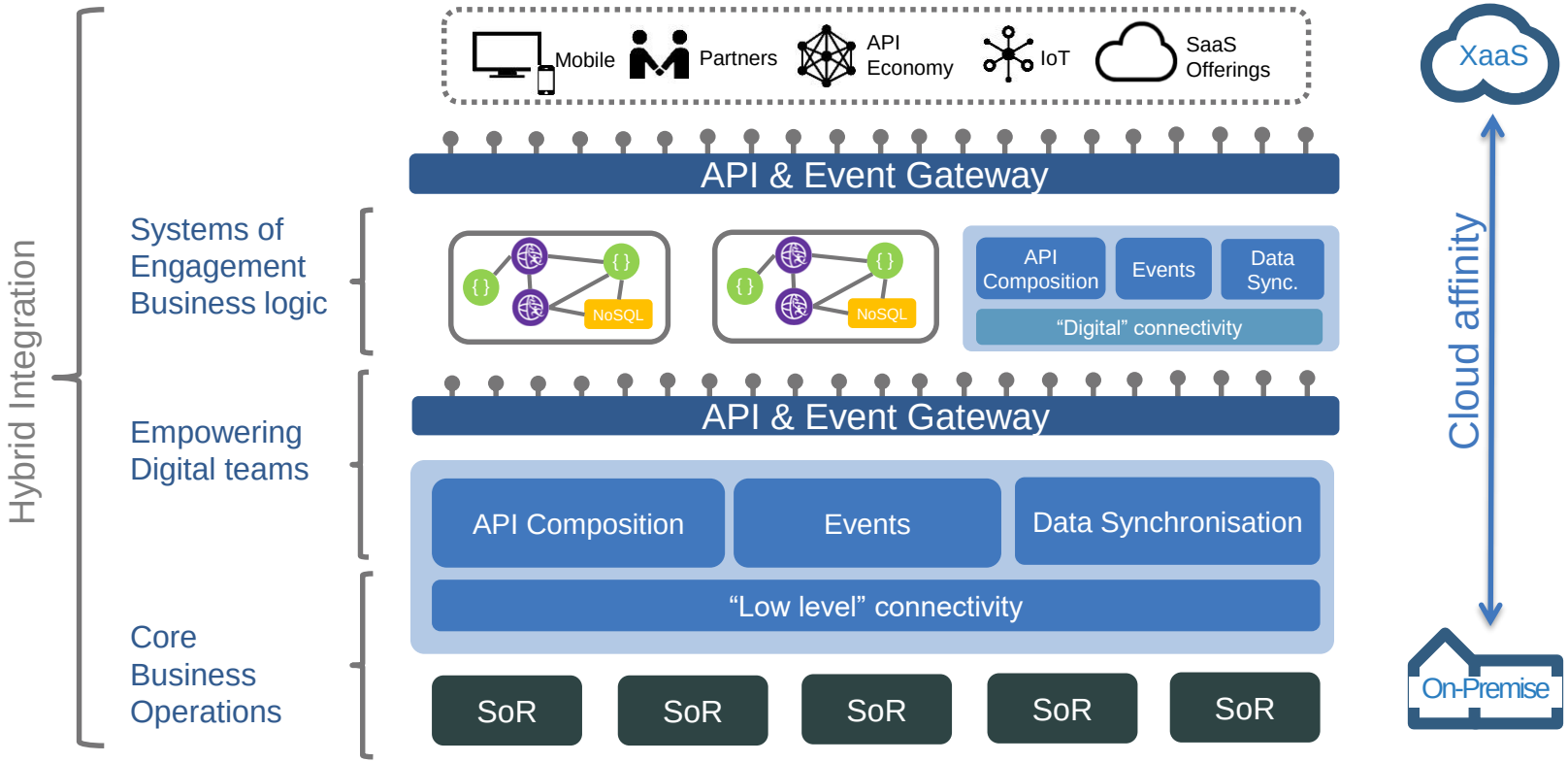
“Integration Design and Architecture”

blog posts on IBM Integration blog:
https://developer.ibm.com/integration/blog/tag/integration-design-and-architecture

The Twelve Factors

I. Codebase
One codebase tracked in revision control, many deploys
II. Dependencies
Explicitly declare and isolate dependencies
III. Config
Store config in the environment
IV. Backing services
Treat backing services as attached resources
V. Build, release, run
Strictly separate build and run stages
VI. Processes
Execute the app as one or more stateless processes
VII. Port binding
Export services via port binding
VIII. Concurrency
Scale out via the process model
IX. Disposability
Maximize robustness with fast startup and graceful shutdown
X. Dev/prod parity
Keep development, staging, and production as similar as possible
XI. Logs
Treat logs as event streams
XII. Admin processes
Run admin/management tasks as one-off processes

Hybrid Integration Reference Architecture



article
http://ibm.biz/HybridIntRefArch

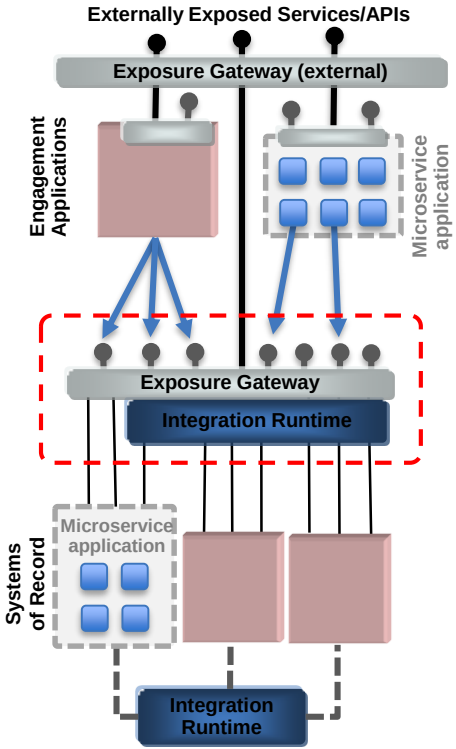
video
http://ibm.biz/HybridIntRefArchYouTube

Introduction of microservices into large enterprises

Microservices architecture provides an alternative way to implement applications within the enterprise, with the aim of providing greater **agility**, **scalability** and **resilience** by breaking the application up into small highly decoupled pieces. These can be safely maintained and scaled independently by smaller teams who can focus on a much smaller scope of functionality.

Note that microservices architecture is about how we structure the internals of an application. From the outside of an application boundary is should be impossible to tell how it is implemented. Microservices architecture says nothing about how we should handle integration between applications across a large enterprise.

Communication between microservice components *within* an application boundary may use no gateway or integration at all. However, when communicating from one application to another *across* application boundaries we would expect to see as a minimum an API gateway to enable discovery, implement appropriate security, protect the provider systems and provide consistent analytics. There may be a need for deeper integration too.



Public API
Enterprise API
API Gateway
Lightweight language runtime
Integration runtime
Request/response integration
Asynchronous integration
Microservice application boundary
ESB pattern?

In theory, this could be described as “all requests go through the ESB”, but this causes confusion, as some will interpret it as everything going through the Integration Hub, where in fact the only mandatory component of the ESB pattern is the Exposure Gateway. This is a critically important distinction because to administer the exposure of an API on the gateway is something that an application team should be able to do directly, with no need to involve a separate team of integration specialists.

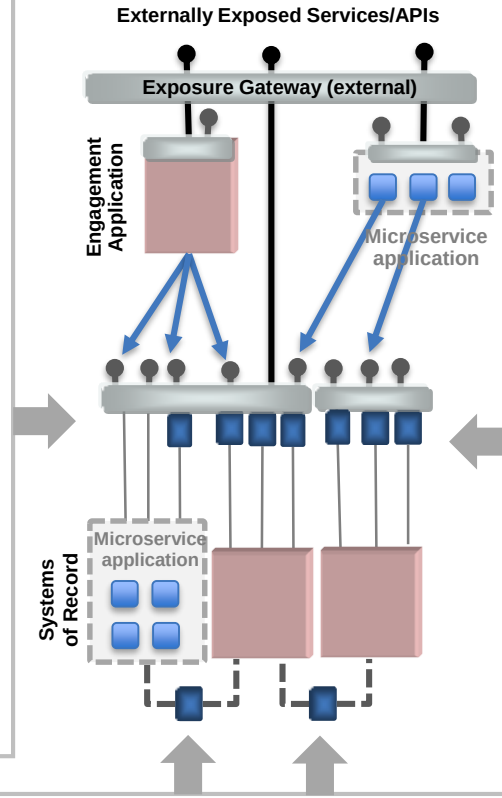
The Integration Hub should only be used where it is needed, such as for deep connectivity, complex composition of downstream requests, detailed data mapping, transactionality, handling of asynchronous transports and so on. If all that is needed is exposing and routing of a synchronous API, a gateway is sufficient.

Fine-grained integration: Containerization of the integration hub

Modern integration runtimes have become more lightweight, and there is a range of more flexible infrastructure including virtual machines, containers. There is no longer an imperative to run all integrations in one runtime.

There is no reason why the centralized ESB pattern can not be broken up into smaller pieces, each containing just a handful of integrations. These can be run in containers such as Docker and then be managed and scaled independently using for example Kubernetes orchestration technology.

This could certainly be seen to be borrowing from microservices principles, even if it is not necessarily full microservices application architecture.



Public API
Enterprise API
API Gateway
Lightweight language runtime
Lightweight **integration** runtime
Request/response integration
Asynchronous integration
ESB pattern??
Microservice application boundary

Would this still be classed as fulfilling the ESB pattern? Does anybody care? The reality is that it still achieves the same end – exposing services in a standardized way – but in a much more agile, scalable and resilient fashion. However, the term ESB's use is sufficiently broken, that it may be best to leave it behind.

Note that pre-ESB asynchronous hub and spoke integration can also be broken up in this way with the same benefits

Decentralized integration

Moving ownership to application teams, simplifying the process

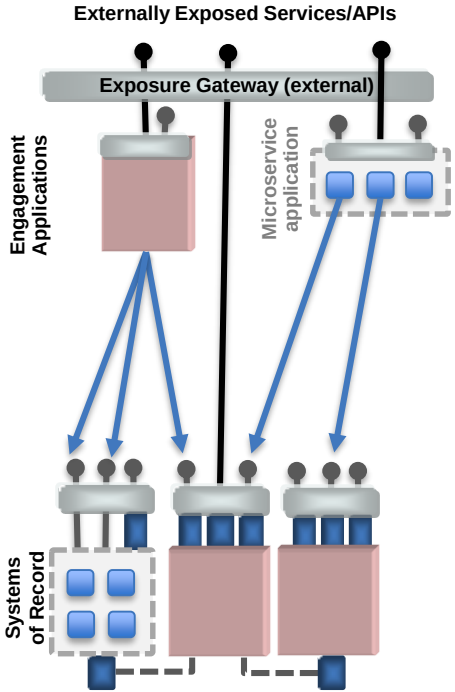
Having broken up the problem into smaller components, and with modern integration tooling becoming much easier to install and use, it is also now easier distribute the work.

Note there is little technical different between this and the previous diagram, the change is more about ownership.

Standardized exposure could now be “owned” by the same team that own the system of record. This significantly improves agility by reducing the number of teams that need to be coordinated for changes to happen.

We call this style “***decentralized integration***” to note that it removes the need for a centralized integration team that can easily become a bottleneck for projects – one of the most common complaints in SOA initiatives.

This is particularly powerful when “moving to cloud”. Integration is already implemented in a cloud friendly way and aligned with the systems of record. When they move to cloud, the deployment follows a more cloud ready path.



Public API

Enterprise API

API Gateway

Lightweight language runtime

Lightweight **integration** runtime

Request/response integration

Asynchronous integration

Microservice application boundary

Note that although ownership of the gateways and the integrations is distributed, that doesn’t necessarily mean each application team must have separate infrastructure. In the diagram, we don’t necessarily have 5 separate gateways for example, there could be just one gateway, or perhaps one for all internal API exposure and one for external.

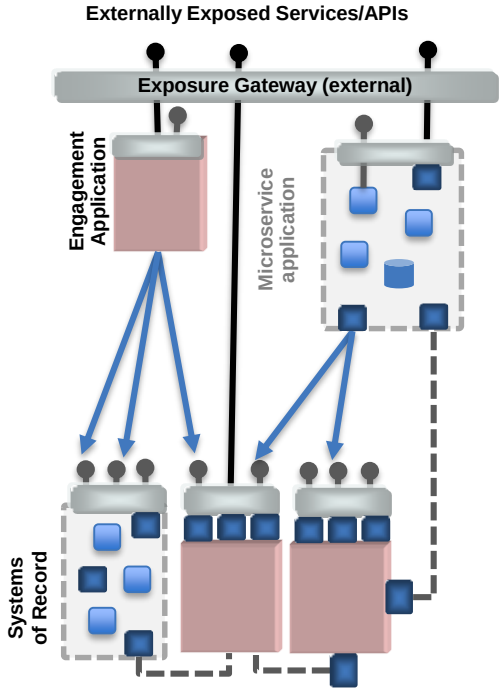
An excellent example of this decentralized ownership living on a potentially centralized infrastructure is API management. Teams completely own and administer their own APIs, but they commonly do that on a multi-tenanted API management infrastructure, sharing an API gateway.

Using lightweight integration runtimes within microservices applications

With only application centric integration, microservices will need to do increasingly more complex integration to take on the compositional logic that was in the past implemented (perhaps incorrectly) in the centralized ESB.

Whilst this integration can be done in raw code, we will quickly end up re-inventing the integration engines that already exist. It would more make sense to use modern lightweight integration runtimes where the task is obviously suited to them.

Typical examples of what would be better done in an integration runtime would be composition/aggregation, complex data mapping, and ingestion of event streams to populate microservices local data stores. Add to that the fact that the integration runtime has pre-built connectors to a vast array of protocols and applications, and sophisticated tooling for parsing an serializing a wide range of data formats.



Public API
Enterprise API
API Gateway
Lightweight language runtime
Lightweight **integration** runtime
Request/response integration
Asynchronous integration
Microservice application boundary

It is interesting that the need for completely decoupled data in the microservices environments is in fact driving a return the use of event centric application design patterns. Messaging and asynchronous interaction in general is something that integration engines are also particularly well suited to.

Micro services Inter-communication

Aim is decoupling for robustness

Messaging where possible
- Lightweight messaging (e.g. AMQP, Kafka)
- Publish/subscribe
- Eventual consistency

Direct calls where necessary
Lightweight protocols
(e.g. JSON/HTTP)
- Load balancing/scaling via service discovery
- Circuit breaker
- Caching

